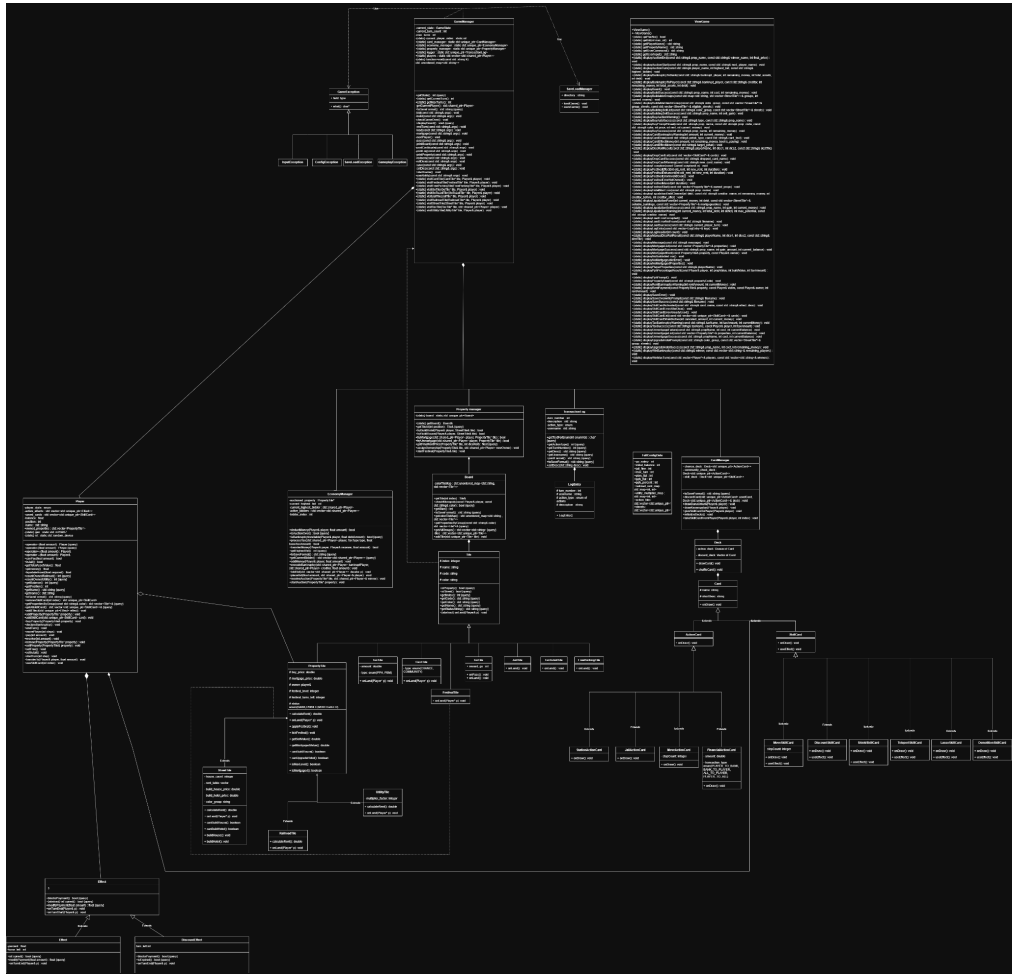
Laporan M2 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

Kode Kelompok : TSR
Nama Kelompok : The Rostschilds

1. 13524024 / Billie Bhaskara Wiba
2. 13524058 / Muhammad Naufal Romi Annafi
3. 13524072 / I Gusti Ngurah Alit Dharma Yudha
4. 13524082 / Zeki Amani
5. 13524083 / Mahmudia Kimdaro Amin

TSR - The Rostschilds

1. Diagram Kelas



Gambar 1.1 Diagram Kelas

https://drive.google.com/file/d/1Mv87YJ5D236dvYIJWsM8QSD1c7X3lmtd/view?usp=sharing

2. Deskripsi Kelas

1. Tile

1.1. Deskripsi

Tile merupakan kelas dasar yang merepresentasikan setiap petak pada board permainan. Kelas ini menyediakan struktur umum serta mendefinisikan perilaku dasar yang akan diimplementasikan secara spesifik oleh kelas turunannya.

1.2. Tanggung Jawab

1.2.1. Menyediakan identitas dasar petak melalui atribut name dan code agar setiap petak dapat dikenali dalam permainan.

1.2.2. Mendefinisikan kontrak polimorfisme melalui method onLand(), sehingga setiap turunan Tile dapat memberikan efek berbeda ketika pemain berhenti di petak tersebut.

1.2.3. Mendukung interaksi antara pemain dan board dengan menjadi titik eksekusi efek permainan.

1.3. Relasi

1.3.1. Board (Composition): Board memiliki kumpulan Tile yang membentuk keseluruhan peta permainan.

1.3.2. CardTile, TaxTile, FestivalTile, PropertyTile (Inheritance): Kelas-kelas tersebut merupakan turunan dari Tile yang mengimplementasikan perilaku spesifik dari method onLand.

1.3.3. Player (Dependency): Tile menggunakan informasi Player untuk menentukan efek yang diberikan, seperti pembayaran, perpindahan, atau efek lainnya.

1.3.4. GameManager (Dependency): Tile berinteraksi dengan GameManager untuk mengubah state permainan secara global, seperti memindahkan pemain atau memicu event tertentu.

1.4. Atribut

1.4.1. index: lokasi tile dalam papan permainan

1.4.2. name: nama lengkap petak untuk ditampilkan ke layar

1.4.3. code: kode singkat petak untuk identifikasi

1.4.4. color: warna tile

1.5. Metode

1.5.1. getIndex(): mengembalikan indeks petak

1.5.2. getName(): mengembalikan nama petak

1.5.3. getCode(): mengembalikan kode petak

1.5.4. getColor(): mengembalikan warna petak

1.5.5. onLand(): method abstrak yang akan diimplementasikan oleh kelas turunan untuk memberikan efek ketika pemain berhenti di petak

2. CardTile

2.1. Deskripsi

CardTile merupakan turunan dari Tile yang merepresentasikan petak yang memicu pengambilan kartu seperti Chance dan Community Chest. Ketika pemain berhenti pada petak ini, pemain akan mengambil kartu dari deck yang sesuai dan menjalankan efek yang terdapat pada kartu tersebut.

2.2. Tanggung Jawab

2.2.1. Menentukan jenis kartu yang akan diambil pemain melalui atribut type (Chance atau Community Chest).

2.2.2. Menghubungkan mekanisme Tile dengan sistem draw kartu menggunakan Deck dalam permainan.

2.2.3. Memicu pengambilan dan eksekusi efek kartu saat pemain berhenti di petak ini melalui method onLand().

2.2.4. Mendukung variasi event permainan melalui integrasi dengan berbagai jenis kartu yang memiliki efek berbeda.

2.3. Relasi

2.3.1. Tile (Inheritance): CardTile merupakan turunan dari Tile yang mengimplementasikan perilaku spesifik saat pemain berhenti di petak.

2.3.2. Card (Dependency): CardTile berinteraksi dengan Card untuk menjalankan efek yang diperoleh pemain setelah mengambil kartu.

2.3.3. Deck (Dependency): CardTile menggunakan CardDeck untuk mengambil kartu sesuai dengan jenisnya (Chance atau Community).

2.3.4. Player (Dependency): CardTile memicu perubahan state pada Player berdasarkan efek kartu yang diambil.

2.3.5. GameManager (Dependency): CardTile dapat berinteraksi dengan GameManager untuk mengatur alur permainan setelah efek kartu dijalankan.

2.4. Atribut

2.4.1. type : enum {CHANCE, COMMUNITY} untuk menentukan jenis kartu yang akan diambil pemain

2.5. Metode

2.5.1. getType(): mengambil type kartu

2.5.2. onLand(): mengambil kartu dari CardDeck sesuai type dan menjalankan efek kartu terhadap pemain

3. TaxTile

3.1. Deskripsi

TaxTile merupakan turunan dari Tile yang merepresentasikan petak yang mewajibkan pemain untuk membayar pajak ketika berhenti di petak tersebut. Pajak yang dikenakan dapat berupa jenis yang berbeda, seperti PPH atau PBM, dengan jumlah tertentu yang memengaruhi kondisi finansial pemain.

3.2. Tanggung Jawab

3.2.1. Menentukan jenis pajak yang dikenakan kepada pemain melalui atribut type.

3.2.2. Menyimpan jumlah pajak yang harus dibayarkan melalui atribut amount.

3.2.3. Mengurangi saldo pemain saat berhenti di petak ini melalui method onLand().

3.3. Relasi

3.3.1. Tile (Inheritance): TaxTile merupakan turunan dari Tile yang mengimplementasikan perilaku pembayaran pajak saat pemain berhenti di petak.

3.3.2. Player (Dependency): TaxTile memodifikasi state Player, khususnya balance, dengan mengurangi sejumlah amount.

3.3.3. GameManager (Dependency): TaxTile dapat berinteraksi dengan GameManager untuk menangani kondisi khusus, seperti pemain tidak mampu membayar (potensi bangkrut) atau pencatatan transaksi.

3.4. Atribut

3.4.1. type : enum {PPH, PBM} untuk menentukan jenis pajak yang dikenakan

3.4.2. amount : jumlah pajak yang harus dibayarkan pemain

3.5. Metode

3.5.1. getTaxAmount(): mengambil atribut amount dari taxTile

3.5.2. onLand(): mengurangi saldo pemain sebesar amount dan memicu penanganan lebih lanjut jika pemain tidak mampu membayar

4. FestivalTile

4.1. Deskripsi

FestivalTile merupakan turunan dari Tile yang merepresentasikan petak aksi khusus yang dapat meningkatkan harga sewa properti milik pemain. Ketika pemain berhenti di petak ini, pemain dapat memilih salah satu properti yang dimilikinya untuk mendapatkan efek peningkatan harga sewa secara sementara dengan mekanisme stacking dan durasi tertentu.

4.2. Tanggung Jawab

4.2.1. Memicu pemilihan properti milik pemain yang akan diberikan efek festival.

4.2.2. Menerapkan efek peningkatan harga sewa (multiplikatif) pada properti yang dipilih.

4.2.3. Mengatur reset durasi efek jika properti yang sama dipilih kembali.

4.2.4. Membatasi jumlah maksimum peningkatan (maksimal 3 kali lipat efek → total 8x harga awal).

4.2.5. Mendukung mekanisme strategis permainan dengan memberikan opsi peningkatan pendapatan pemain secara temporer.

4.3. Relasi

4.3.1. Tile (Inheritance): FestivalTile merupakan turunan dari Tile yang mengimplementasikan perilaku khusus saat pemain berhenti di petak.

4.3.2. Player (Dependency): FestivalTile menggunakan data properti milik Player untuk menentukan target efek.

4.3.3. PropertyTile (Dependency): FestivalTile memodifikasi state dari PropertyTile, khususnya terkait harga sewa dan durasi efek festival.

4.3.4. GameManager (Dependency): FestivalTile dapat berinteraksi dengan GameManager untuk mengelola durasi efek tiap giliran dan sinkronisasi state permainan.

4.4. Atribut

Tidak ada atribut langsung pada FestivalTile karena state efek disimpan pada PropertyTile masing-masing

4.5. Metode

4.5.1. onLand(): memungkinkan pemain memilih salah satu properti miliknya, lalu menerapkan atau memperbarui efek peningkatan harga sewa beserta durasinya

4.6. PropertyTile

4.7. Deskripsi

PropertyTile merupakan turunan dari Tile yang merepresentasikan petak yang dapat dimiliki oleh pemain. Kelas ini menjadi dasar bagi seluruh jenis properti (Street, Railroad, Utility) dan mengatur aspek umum seperti kepemilikan, status, serta interaksi ekonomi dalam permainan.

4.8. Tanggung Jawab

4.8.1. Menyimpan informasi kepemilikan properti melalui atribut owner.

4.8.2. Mengatur status properti (BANK, OWNED, MORTGAGED) yang memengaruhi perilaku permainan.

4.8.3. Menyediakan mekanisme dasar perhitungan sewa yang akan diimplementasikan secara spesifik oleh kelas turunan.

4.8.4. Mengelola efek tambahan seperti Festival yang memodifikasi nilai sewa secara temporer.

4.8.5. Menangani interaksi dasar ketika pemain berhenti di properti melalui method onLand().

4.9. Relasi

4.9.1. Tile (Inheritance): PropertyTile merupakan turunan dari Tile.

4.9.2. Player (Association): PropertyTile dimiliki oleh Player sebagai owner.

4.9.3. GameManager (Dependency): PropertyTile dapat berinteraksi dengan GameManager untuk transaksi, lelang, atau kondisi khusus.

4.9.4. FestivalTile (Dependency): PropertyTile menerima efek peningkatan sewa dari FestivalTile.

4.10. Atribut

4.10.1. buy_price : harga beli properti

4.10.2. mortgage_price : nilai gadai properti

4.10.3. owner : pemilik properti

4.10.4. festival_level : tingkat efek festival (0–3)

4.10.5. festival_turns_left: sisa durasi efek festival

4.10.6. status : enum {BANK, OWNED, MORTGAGED} untuk status properti

4.10.7. type: enum {STREET, RAILROAD, UTILITY} untuk tipe properti

4.11. Metode

4.11.1. virtual calculateRent() : menghitung biaya sewa berdasarkan kondisi properti

4.11.2. onLand() : menangani pembelian, pembayaran sewa, atau lelang

4.11.3. applyFestival() : meningkatkan efek festival dan mereset durasi

4.11.4. decreaseFestival() : mengurangi durasi efek festival setiap giliran

4.11.5. getBuyPrice() : menghitung nilai beli properti

4.11.6. getMortgageValue() : menghitung nilai gadai properti

4.11.7. getPropertyStatus(): mengambil status properti

4.11.8. setPropertyStatus(): mengubah state status properti saat ini

4.11.9. getPropertyOwner(): mengambil pointer Player pemilik properti

4.11.10. setPropertyOwner(): mengubah pemilik properti

4.11.11. virtual getPropertyType(): berupa virtual, mengambil tipe dari properti

5. StreetTile

5.1. Deskripsi

StreetTile merupakan turunan dari PropertyTile yang merepresentasikan properti umum dengan sistem pembangunan dan color group. StreetTile memungkinkan pemain untuk membangun rumah dan hotel guna meningkatkan nilai sewa.

5.2. Tanggung Jawab

5.2.1. Mengelola pembangunan rumah dan hotel pada properti.

5.2.2. Menentukan biaya sewa berdasarkan jumlah bangunan dan status monopoli.

5.2.3. Mendukung aturan pemerataan pembangunan dalam satu color group.

5.2.4. Mengintegrasikan efek Festival ke dalam perhitungan sewa akhir.

5.3. Relasi

5.3.1. PropertyTile (Inheritance): Street merupakan turunan dari PropertyTile.

5.3.2. Player (Dependency): digunakan untuk mengecek kepemilikan dan monopoli color group.

5.3.3. Board (Dependency): digunakan untuk mengecek properti lain dalam color group yang sama.

5.4. Atribut

5.4.1. building_level: jumlah rumah dalam sebuah tile, jika jumlah rumah==5 maka sama dengan 1 hotel

5.4.2. rent_list (vector): tabel sewa berdasarkan level bangunan

5.4.3. build_house_price : harga membangun rumah

5.4.4. build_hotel_price : harga membangun hotel

5.5. Metode

5.5.1. calculateRent() : menghitung sewa berdasarkan bangunan, monopoli, dan efek festival

5.5.2. onLand() : menangani pembelian atau pembayaran sewa

5.5.3. getPropertyType(): mengambil jenis properti

5.5.4. getBuildingLevel(): mengambil jumlah bangunan dalam tile tersebut

5.5.5. canBuildHouse() : mengembalikan apakah player bisa membangun rumah

5.5.6. canBuildHotel() : mengembalikan apakah player bisa membangun hotel

5.5.7. canSellBuilding(): mengembailkan apakah player bisa menjual bangunan

5.5.8. buildHouse() : membangun rumah sesuai aturan

5.5.9. buildHotel() : upgrade ke hotel

5.5.10. sellBuilding(): menjual bangunan

5.5.11. getHousePrice(): mengambil nilai harga untuk membangun rumah

5.5.12. getHotelPrice(): mengambil nilai harga untuk membangun hotel

6. RailroadTile

6.1. Deskripsi

RailroadTile merupakan turunan dari PropertyTile yang memberikan mekanisme kepemilikan otomatis tanpa pembelian. Nilai sewa bergantung pada jumlah RailroadTile yang dimiliki oleh pemain.

6.2. Tanggung Jawab

6.2.1. Menangani kepemilikan otomatis saat pertama kali diinjak.

6.2.2. Menghitung biaya sewa berdasarkan jumlah Railroad yang dimiliki pemain.

6.2.3. Mendukung interaksi ekonomi seperti gadai dan penjualan.

6.3. Relasi

6.3.1. PropertyTile (Inheritance) : Railroad merupakan turunan dari PropertyTile.

6.3.2. Player (Dependency) : digunakan untuk menghitung jumlah Railroad yang dimiliki.

6.4. Atribut

railroad_multiplier: berupa map untuk memetakan harga sewa dari tile railroad berdasarkan key_value berupa jumlah tile railroad yang dimiliki player

6.5. Metode

6.5.1. calculateRent() : menghitung sewa berdasarkan jumlah Railroad milik owner

6.5.2. onLand() : memberikan kepemilikan otomatis atau memproses pembayaran sewa

6.5.3. getPropertyType() : mengambil tipe properti

7. UtilityTile

7.1. Deskripsi

UtilityTile merupakan turunan dari PropertyTile yang memiliki mekanisme sewa berbasis hasil lemparan dadu. Nilai sewa dihitung dari angka dadu pemain yang mendarat dikalikan faktor tertentu.

7.2. Tanggung Jawab

7.2.1. Menangani kepemilikan otomatis tanpa pembelian.

7.2.2. Menghitung sewa berdasarkan hasil dadu dan jumlah Utility yang dimiliki owner.

7.2.3. Mendukung variasi perhitungan sewa berbasis konfigurasi permainan.

7.3. Relasi

7.3.1. PropertyTile (Inheritance): Utility merupakan turunan dari PropertyTile.

7.3.2. Player (Dependency): digunakan untuk menentukan jumlah Utility yang dimiliki.

7.4. Atribut

7.4.1. utility_multiplier : faktor pengali sewa

7.5. Metode

7.5.1. calculateRent() : menghitung sewa berdasarkan nilai dadu

7.5.2. onLand() : memberikan kepemilikan atau memproses pembayaran

7.5.3. getPropertyType() : mengambil tipe properti

8. GoTile

8.1. Deskripsi

GoTile merepresentasikan petak awal permainan (GO) tempat pemain memulai permainan. Pemain yang melewati atau mendarat di petak ini akan mendapatkan sejumlah uang dari bank.

8.2. Tanggung Jawab

8.2.1. Memberikan reward kepada pemain saat melewati atau mendarat

8.2.2. Menjadi titik awal posisi pemain

8.3. Atribut

8.3.1. reward_go : jumlah uang yang diberikan saat melewati/berhenti

8.4. Metode

8.4.1. onPass() : dipanggil saat pemain melewati GO

8.4.2. getReward() : mengambil jumlah uang yang didapat jika melewati go

8.4.3. onLand() : dipanggil saat pemain berhenti di GO

9. JailTile

9.1. Deskripsi

JailTile merepresentasikan petak penjara. Pemain yang hanya mendarat di petak ini tidak otomatis masuk penjara, tetapi hanya berkunjung.

9.2. Tanggung Jawab

9.2.1. Menjadi representasi lokasi penjara di board

9.2.2. Menjadi referensi posisi saat pemain benar-benar dipenjara

9.3. Atribut

9.3.1. jail_fine: jumlah uang yang harus dibayar saat masuk penjara

9.4. Metode

9.4.1. onLand() : netral hanya menampilkan player melewati petak ini

10. FreeParkingTile

10.1. Deskripsi

FreeParkingTile merupakan petak netral yang tidak memberikan efek apa pun kepada pemain yang mendarat di atasnya.

10.2. Tanggung Jawab

10.2.1. Menjadi petak tanpa efek

10.3. Atribut

Tidak Ada

10.4. Metode

10.4.1. OnLand() : tidak melakukan apa pun

11. GoToJailTile

11.1. Deskripsi

GoToJailTile merupakan petak yang memaksa pemain untuk langsung masuk ke penjara tanpa melewati GO dan tanpa menerima reward.

11.2. Tanggung Jawab

11.2.1. Memindahkan pemain ke JailTile

11.2.2. Mengubah state pemain menjadi dalam penjara

11.3. Atribut

Tidak ada

11.4. Metode

11.4.1. onLand() : memindahkan pemain ke penjara

12. Card

12.1. Deskripsi

12.1.1. Kelas dasar yang merepresentasikan segala kartu dalam permainan. Menyediakan struktur umum serta deklarasi *virtual method* untuk *method* yang diekspektasikan dimiliki oleh *cards.*

12.2. Tanggung Jawab

12.2.1. Menyediakan identitas dari setiap *cards.*

12.2.2. Mendefinisikan kontrak polimorfisme dengan *method* useEffect() dengan ekspektasi tiap turunan memberikan definisi masing-masing terhadap useEffect()

12.3. Relasi

12.3.1. ActionCard, SkillCard (inheritance): kelas-kelas tersebut merupakan turunan dari kelas ini dan mendefinisikan *virtual method* yang dimiliki kelas ini.

12.4. Atribut

12.4.1. name: nama dari kartu. Diperlukan baik oleh SkillCard maupun ActionCard untuk save.

12.4.2. random_number_generator : sebagai helper untuk mengambil kartu secara acak

12.5. Metode

12.5.1. virtual onDraw(): metode abstrak yang didefinisikan oleh kelas turunannya. Metode ini dipanggil ketika kartu diambil dari deck

12.5.2. virtual toSaveFormat(): metode abstrak untuk save yang didefinisikan kelas turunannya. Dipanggil waktu save

13. ActionCard

13.1. Deskripsi

Merupakan kelas dari kartu-kartu yang tidak dapat disimpan dan hanya melakukan suatu aksi saat didapatkan itu juga.

13.2. Tanggung Jawab

13.2.1. Merepresentasikan kartu yang tidak dapat disimpan

13.2.2. Memberikan polimorfisme dari card dan menurunkannya untuk kartu yang lebih spesifik.

13.3. Relasi

13.3.1. Card (inheritance): turunan abstract class dari card dan menurunkan polimorfisme ke kelas turunannya.

13.4. Atribut

13.5. Metode

13.5.1. toSaveFormat(): Untuk kartu action, hanya nama yang ditulis

14. StationActionCard

14.1. Deskripsi

Merupakan representasi kelas dari kartu kesempatan yang menempatkan pemain ke stasiun terdekat.

14.2. Tanggung Jawab

14.2.1. Mengubah state Player (posisi) dari *state* semula ke stasiun terdekat

14.3. Relasi

14.3.1. ActionCard (inheritance): kelas ini merupakan kelas turunan dari ActionCard dan mendefinisikan *abstract method* onDraw() dari ActionCard.

14.4. Atribut

14.5. Metode

14.5.1. onDraw(): menentukan stasiun terdekat dan memindahkan posisi player ke stasiun tersebut

15. JailActionCard

15.1. Deskripsi

Merupakan kelas dari kartu kesempatan yang menempatkan pemain di dalam penjara

15.2. Tanggung Jawab

Mengubah state player posisi dari state semula ke stasiun terdekat

15.3. Relasi

15.3.1. ActionCard (inheritance): JaliActionCard merupakan kelas turunan dari ActionCard dan mendefinisikan *abstract method* onDraw() dari ActionCard.

15.4. Atribut

15.5. Metode

15.5.1. onDraw(): mengubah *state player* dan menempatkannya di penjara

16. MoveActionCard

16.1. Deskripsi

Representasi kelas dari katu *action* yang memindahkan player beberapa langkah sesuai dengan stepCount

16.2. Tanggung Jawab

16.2.1. Mengelola *state player* (lokasi) sesuai dengan jumlah perpindahan yang ditentukan

16.3. Relasi

16.3.1. ActionCard: kelas ini merupakan turunan dari ActionCard dan mendefinisikan *abstract method* onDraw() dari ActionCard.

16.4. Atribut

16.4.1. walkDistance : Jumlah langkah yang dilakukan dalam pergerakan

16.4.2. random_number_generator : sebagai helper untuk menentukan jumlah langkah secara acak

16.5. Metode

16.5.1. onDraw() : menentukan jumlah pergerakan dan mengubah lokasi player sesuai dengan pergerakan yang diberikan

17. FinancialActionCard

17.1. Deskripsi

Kartu yang efeknya melakukan aksi finansial terhadap player dari bank atau player ke player lainnya

17.2. Tanggung Jawab

17.3. Relasi

17.3.1. ActionCard: kelas ini merupakan turunan dari ActionCard dan mendefinisikan *abstract method* onDraw() dari ActionCard.

17.4. Atribut

17.4.1. amount: jumlah uang yang diberikan dalam aksi finansial

17.4.2. transaction_type: enum yang yang merepresentasikan jenis transaksi, seperti PLAYER_TO_BANK, BANK_TO_PLAYER, ALL_TO_PLAYER, dan PLAYER_TO_ALL

17.5. Metode

17.5.1. onDraw(): melakukan aksi finansial sesuai dengan enum transaction_type yang didapatkan

18. SkillCard

18.1. Deskripsi

Merepresentasikan kartu yang bisa disimpan untuk digunakan di lain waktu.

18.2. Tanggung Jawab

18.2.1. Sebagai kelas umum untuk merepresentasikan semua kartu skill yang dapat disimpan

18.2.2. Menurunkan polimorfisme dari card ke semua kartu skill

18.3. Relasi

18.3.1. Card (inheritance): turunan abstract class dari card dan menurunkan polimorfisme ke kelas turunannya

18.4. Atribut

18.4.1. name : nama dari kartu berupa string

18.5. Metode

18.5.1. Virtual onDraw(): metode abstrak yang didefinisikan oleh kelas turunannya. Metode ini dipanggil ketika kartu diambil dari deck

18.5.2. Virtual UseEffect(): metode abstrak yang didefinisikan oleh kelas turunannya. Metode ini dipanggil ketika user ingin mengaktifkan efek dari kartu

18.5.3. virtual toSaveFormat() : mengembalikan string dalam format save file untuk objek class ini atau untuk turunan dari kelas ini yang tidak memiliki atribut

19. MoveSkillCard

19.1. Deskripsi

Representasi kelas dari kartu skill *move* yang memungkinkan player untuk bergerak maju beberapa petak yang ditentukan secara acak saat mendapatkan kartunya.

19.2. Tanggung Jawab

19.2.1. Menghitung secara acak atas jumlah langkah yang didapatkannya

19.2.2. Menangani *state* posisi dari *player* dan mengubahnya sesuai dengan jumlah pergerakan yang didapatkan

19.3. Relasi

19.3.1. SkillCard (inheritance): kelas ini merupakan turunan dari SkillCard dan memberikan definisi terhadap virtual methods onDraw() dan useEffect()

19.4. Atribut

19.4.1. step : jumlah step yang dilakukan saat mengaktifkan card ini

19.5. Metode

19.5.1. onDraw() : menghitung steps yang didapatkan saat player mendapatkan kartu ini dan menempatkan kartu ke tangan *player*

19.5.2. useEffect() : memajukan player sesuai dengan atribut stepCount

19.5.3. toSaveFormat() : mengembalikan string dalam format save file untuk objek class ini

20. DiscountSkillCard

20.1. Deskripsi

Representasi kelas dari kartu skill *discount* yang memungkinkan player untuk memberikan diskon secara acak kepada player

20.2. Tanggung Jawab

20.2.1. Menghitung diskon yang didapatkan saat mendapatkan kartu

20.2.2. Mengubah harga dari suatu aksi sesuai dengan diskon yang didapatkan

20.3. Relasi

20.3.1. SkillCard (inheritance): kelas ini merupakan turunan dari SkillCard dan memberikan definisi terhadap virtual methods onDraw() dan useEffect()

20.4. Atribut

20.4.1. percentage : persentase diskon yang digunakan saat kartu digunakan

20.5. Metode

20.5.1. onDraw(): menghitung secara acak jumlah diskon yang didapatkan saat menggunakan kartu ini dan menempatkan kartu ke tangan *player*

20.5.2. useEffect(): mengurangi harga dari suatu aksi sesuai dengan diskon yang diberikan

20.5.3. toSaveFormat() : mengembalikan string dalam format save file untuk objek class ini

21. ShieldSkillCard

21.1. Deskripsi

Representasi kelas dari kartu yang dapat memberikan player sebuah perlindungan dari tagihan atau sanksi

21.2. Tanggung Jawab

21.2.1. Meng-*override effect* dari sanksi atau tagihan apapun yang diberikan ke *player* pada *turn* itu

21.3. Relasi

21.3.1. SkillCard (inheritance): kelas ini merupakan turunan dari SkillCard dan memberikan definisi terhadap virtual methods onDraw() dan useEffect()

21.4. Atribut

21.5. Metode

21.5.1. onDraw(): menempatkan kartu ke tangan *player*

21.5.2. useEffect: meng-*override* semua *effect* negatif yang dilakukan ke player pada turn yang sama saat kartu diaktifkan

22. TeleportSkillCard

22.1. Deskripsi

Representasi kelas dari kartu *skill* yang memungkinkan *player* untuk berpindah kemanapun di *board*

22.2. Tanggung Jawab

22.2.1. Mengelola *state player* (posisi) untuk memindahkannya sesuai dengan pilihan *player*

22.3. Relasi

22.3.1. SkillCard: kelas ini merupakan turunan dari SkillCard dan memberikan definisi terhadap virtual methods onDraw() dan useEffect()

22.4. Atribut

22.5. Metode

22.5.1. onDraw(): menempatkan kartu ke tangan player

22.5.2. useEffect(): memberikan *player* pilihan untuk berpindah tempat dan memindahkan posisi *player* sesuai dengan pilihannya

23. LassoSkillCard

23.1. Deskripsi

Representasi kelas dari kartu *skill* yang memungkinkan *player* untuk memindahkan *player* lain ke petak yang ditempatinya

23.2. Tanggung Jawab

23.2.1. Mengelola *state player* (posisi) selain *player* yang memainkan kartu ini untuk memindahkannya ke posisi *player* yang memainkan kartunya

23.3. Relasi

23.3.1. SkillCard(inheritance): kelas ini merupakan turunan dari SkillCard dan memberikan definisi terhadap virtual methods onDraw() dan useEffect()

23.4. Atribut

23.5. Metode

23.5.1. onDraw(): menempatkan kartu ke tangan player

23.5.2. useEffect(): memberikan *player* untuk memilih *player* lain yang ditempatkan di petak yang ditempatinya

24. DemolitionSkillCard

24.1. Deskripsi

Representasi kelas dari kartu *skill* yang memungkinkan *player* untuk menghancurkan suatu properti

24.2. Tanggung Jawab

24.2.1. Mengubah state dari PropertyTile (status) untuk menghancurkannya

24.2.2. Mengubah state dari *player* (kepemilikan properti) dengan menghapuskan properti yang dihancurkan

24.3. Relasi

24.3.1. SkillCard: kelas ini merupakan turunan dari SkillCard dan memberikan definisi terhadap virtual methods onDraw() dan useEffect()

24.4. Atribut

24.5. Metode

24.5.1. onDraw(): menempatkan kartu ke tangan player

24.5.2. useEffect(): memberikan player pilihan properti yang dimiliki *player* lain dan menghancurkan properti yang telah dipilih

25. Board

25.1. Deskripsi

Board merepresentasikan papan permainan yang terdiri dari kumpulan Tile yang tersusun dalam urutan tertentu. Kelas ini bertanggung jawab dalam mengelola seluruh petak permainan, termasuk inisialisasi, penyimpanan, serta penyediaan akses

terhadap Tile berdasarkan posisi. Selain itu, Board juga menyediakan mekanisme untuk mengelompokkan properti berdasarkan color group guna mendukung aturan monopoli.

25.2. Tanggung Jawab

25.2.1. Menginisialisasi dan menyimpan seluruh Tile yang membentuk papan permainan.

25.2.2. Menyediakan akses Tile berdasarkan indeks posisi pemain.

25.2.3. Mengelompokkan PropertyTile (khususnya Street) berdasarkan color group.

25.2.4. Mendukung pengecekan kondisi monopoli untuk suatu color group.

25.2.5. Menjadi sumber referensi utama bagi GameManager dalam navigasi pemain di papan

25.3. Relasi

25.3.1. Tile (Composition): Board memiliki sekumpulan Tile yang membentuk struktur papan permainan.

25.3.2. PropertyTile (Dependency): Board mengelompokkan PropertyTile berdasarkan color group untuk mendukung mekanisme monopoli.

25.3.3. Player (Dependency): Board digunakan oleh Player (melalui GameManager) untuk menentukan posisi dan pergerakan di papan.

25.3.4. GameManager (Dependency): Board digunakan oleh GameManager untuk mengatur alur permainan, seperti perpindahan pemain dan interaksi dengan Tile.

25.3.5. Config (Dependency): Config digunakan untuk membuat Board.

25.4. Atribut

25.4.1. tiles (vector berisi Tile) : daftar Tile yang tersusun sesuai urutan papan

25.4.2. colorTileMap (map antara color dan StreetTile) : pemetaan color group ke daftar PropertyTile dalam grup tersebut

25.5. Metode

25.5.1. getTile() : mengembalikan Tile pada posisi tertentu

25.5.2. addTile() : menambahkan Tile ke dalam papan saat inisialisasi

25.5.3. checkMonopoly() : mengecek apakah pemain memiliki seluruh StreetTile dalam color group tertentu

25.5.4. getPropertiesByGroup() : mengembalikan properties yang termasuk dalam satu grup warna

25.5.5. getAllGroups() : mengembalikan grup properti berdasarkan warna

25.5.6. toSaveFormat() : mengembalikan string dalam format save file untuk objek board

26. Deck

26.1. Deskripsi

Deck merepresentasikan kumpulan kartu dalam permainan yang digunakan sebagai sumber pengambilan kartu oleh pemain. Kelas ini mengelola siklus hidup kartu, mulai dari pengambilan (draw), penyimpanan kartu yang telah digunakan (discard), hingga pengacakan ulang kartu untuk menjaga variasi permainan.

26.2. Tanggung Jawab

26.2.1. Menyimpan kumpulan kartu yang siap diambil pemain dalam struktur antrean (drawCards).

26.2.2. Menyimpan kartu yang telah digunakan dalam discard pile.

26.2.3. Menyediakan mekanisme pengambilan kartu secara berurutan.

26.2.4. Mengelola pengacakan ulang kartu ketika deck habis atau diperlukan.

26.2.5. Mendukung berbagai jenis kartu (Chance, Community, Special) melalui penggunaan template class.

26.3. Relasi

26.3.1. Card (Composition): CardDeck memiliki kumpulan objek Card sebagai isi deck.

26.3.2. CardTile (Dependency): CardDeck digunakan oleh CardTile untuk mengambil kartu ketika pemain berhenti di petak terkait.

26.3.3. Player (Dependency): efek dari kartu yang diambil akan memengaruhi state Player.

26.3.4. GameManager (Dependency): CardDeck dapat berinteraksi dengan GameManager untuk sinkronisasi alur permainan, seperti reset deck atau logging.

26.4. Atribut

26.4.1. active_deck (queue of Card) : antrean kartu yang siap diambil

26.4.2. discard_deck (vector of Card) : kumpulan kartu yang telah digunakan

26.5. Metode

26.5.1. drawCard() : mengambil kartu teratas dari drawCards, lalu memindahkannya ke discard

26.5.2. shuffleCard() : mengacak kartu dalam discard dan mengembalikannya ke drawCards ketika deck kosong

26.5.3. addToDiscard() : menambahkan sebuah kartu ke dalam pile discard

26.5.4. toSaveFormat() : mengembalikan string dalam format save file untuk objek class ini

27. Player

27.1. Deskripsi

Player merepresentasikan entitas pemain dalam permainan yang memiliki identitas, posisi, sumber daya, serta aset yang dimiliki. Kelas ini bertanggung jawab dalam mengelola seluruh state yang berkaitan dengan pemain, termasuk pergerakan, kepemilikan properti, penggunaan kartu, dan interaksi dengan elemen permainan lainnya.

27.2. Tanggung Jawab

27.2.1. Menyimpan identitas dan status pemain selama permainan berlangsung.

27.2.2. Mengelola posisi pemain di atas board.

27.2.3. Mengatur saldo (balance) pemain serta transaksi keuangan.

27.2.4. Mengelola kepemilikan properti (pembelian, penjualan, dan efek terkait).

27.2.5. Mengelola kartu kemampuan spesial (SkillCard), termasuk penyimpanan, pemilihan, penggunaan, dan pembuangan kartu.

27.2.6. Menyediakan aksi-aksi yang dapat dilakukan pemain selama giliran berlangsung.

27.2.7. Berinteraksi dengan GameManager untuk menjalankan aksi yang memengaruhi state permainan.

27.3. Relasi

27.3.1. GameManager (Association/Dependency): Player berinteraksi dengan GameManager untuk menjalankan aksi seperti berpindah, transaksi, dan penggunaan kartu.

27.3.2. PropertyTile (Aggregation): Player memiliki kumpulan PropertyTile sebagai aset, namun lifecycle property tetap dikontrol oleh Board.

27.3.3. SkillCard (Aggregation): Player menyimpan kartu kemampuan spesial yang dapat digunakan dalam permainan.

27.3.4. Tile (Dependency): Player berpindah antar Tile dan memicu efek tile melalui GameManager.

27.4. Atribut

27.4.1. name : identitas pemain

27.4.2. balance : jumlah uang yang dimiliki pemain

27.4.3. position : posisi pemain pada board (index tile)

27.4.4. player_state : enum{FREE, INJAIL, BANKCRUPT} untuk menandakan status player saat ini

27.4.5. owned_properties (vector of PropertyTile) : daftar properti yang dimiliki

27.4.6. saved_card (vector of SkillCard) : kartu kemampuan yang dimiliki dengan maksimal 3

27.4.7. active_effects (vector of Effect) : effect yang aktif pada player

27.5. Metode

27.5.1. increaseBalance() : menambah saldo pemain

27.5.2. decreaseBalance() : mengurangi saldo pemain

27.5.3. move() : memindahkan pemain sejumlah langkah tertentu

27.5.4. buyProperty() : membeli properti

27.5.5. sellProperty() : menjual properti

27.5.6. mortgageProperty() : menggadaikan properti

27.5.7. addSkillCard() : menambahkan kartu ke tangan pemain

27.5.8. removeSkillCard() : membuang kartu dari tangan pemain

27.5.9. useCard() : menggunakan kartu (maksimal 1 per giliran)

27.5.10. getTotalAssetValue() : mengembalikan nilai total aset yang dimiliki pemain

27.5.11. resetTurnState() : reset status pemain

27.5.12. canPay() : cek apakah pemain bisa membayar sejumlah uang

27.5.13. liquidateAsset( ) : merubah aset kepemilikan menjadi uang

27.5.14. declareBankruptcy() : mengubah status pemain menjadi bangkrut

28. Effect

28.1.1. Deskripsi

Merepresentasikan efek yang sedang dimiliki player dan menerapkan effect tersebut kepada user sesuai kondisinya.

28.1.2. Tanggung Jawab

28.1.3. Atribute

-

28.1.4. Metode

- virtual void onTurnStart(Player& p): dipanggil ketika turn user dimulai untuk mengubah state
- virtual void onTurnEnd(Player& p);: dipanggil ketika turn user selesai untuk mengubah state
- virtual bool isExpired() const = 0: untuk mengecek apakah efek sudah selesai
- virtual bool blocksPayment() const: untuk effect yang memiliki effect memblokir pembayaran
- virtual float modifyPayment(float amount) const: untuk effect yang memiliki effect mengurangi harga pembayaran

29. ShieldEffect

29.1.1. Deskripsi

Untuk memberikan efek memblokir pembayaran

29.1.2. Tanggung Jawab

- Memberi efek memblokir pembayaran pada user

29.1.3. Attribute

- Turn_left: int

29.1.4. metode

- void onTurnEnd(Player& p) override: dipanggil ketika turn user selesai untuk mengubah state
- bool isExpired() const override:untuk mengecek apakah efek sudah selesai
- bool blocksPayment() const override: dipanggil ketika player perlu membayar sewa agar tidak perlu membayar.

30. DiscountEffect

30.1. Deskripsi

Untuk memberikan efek diskon

30.2. Tanggung Jawab

- Memberi efek mendiskon pembayaran user

30.3. Attribute

- Percentage: float
- Turn_left: int

30.4. metode

- void onTurnEnd(Player& p) override: dipanggil ketika turn user selesai untuk mengubah state
- bool isExpired() const override:untuk mengecek apakah efek sudah selesai
- bool modifyPayment() const override: dipanggil ketika player perlu membayar sewa agar biaya dikurangi.

31. LogEntry

31.1.

31.2. Deskripsi

LogEntry merepresentasikan satu entri log yang mencatat kejadian spesifik dalam permainan. Setiap LogEntry menyimpan informasi terstruktur mengenai aksi yang dilakukan pemain pada suatu turn sehingga dapat digunakan untuk pelacakan, debugging, maupun penampilan riwayat permainan.

31.3. Tanggung Jawab

31.3.1. Menyimpan informasi dasar dari suatu kejadian dalam permainan.

31.3.2. Mewakili satu aksi pemain secara terstruktur (turn, pelaku, jenis aksi, dan detail).

31.3.3. Mendukung pencatatan berbagai jenis event seperti pergerakan, transaksi, penggunaan kartu, dan event khusus lainnya.

31.3.4. Menyediakan representasi data yang dapat ditampilkan kembali kepada pengguna.

31.4. Relasi

31.4.1. TransactionLog (Composition): LogEntry merupakan bagian dari TransactionLog sebagai kumpulan riwayat kejadian permainan.

31.5. Atribut

31.5.1. turn_number : nomor giliran saat aksi terjadi

31.5.2. username : nama pemain yang melakukan aksi

31.5.3. action_type : enum jenis aksi (DADU, SEWA, BELI, KARTU, FESTIVAL, LELANG, GADAI, SAVE, LOAD, dll.)

31.5.4. description : deskripsi detail aksi (misalnya jumlah uang, efek, dll)

31.6. Metode

31.6.1. setDesc(string desc) : mengubah atribut description pada LogEntry

31.6.2. getTurnNumber() : mengambil atribut turn_number

31.6.3. getUsername() : mengambil atribut username

31.6.4. getDesc() : mengambil atribut description

31.6.5. getActionType() : mengambil atribut action_type

31.6.6. printFormat() : fungsi helper untuk mengembalikan string untuk output

31.6.7. toSaveFormat() : fungsi helper untuk mengembalikan string dalam format save file

31.6.8. getTextForEnum(int enumVal) : fungsi helper untuk mengubah enum action_type menjadi string

32. TransactionLog

32.1. Deskripsi

TransactionLog bertanggung jawab dalam mencatat dan mengelola seluruh kejadian signifikan yang terjadi selama permainan berlangsung. Kelas ini bekerja secara otomatis di latar belakang untuk menyimpan riwayat aksi pemain dalam bentuk kumpulan LogEntry yang dapat diakses kembali untuk ditampilkan atau disimpan ke file.

32.2. Tanggung Jawab

32.2.1. Menyimpan seluruh LogEntry selama sesi permainan berlangsung.

32.2.2. Mencatat setiap kejadian penting secara otomatis melalui method pencatatan.

32.2.3. Menyediakan akses terhadap riwayat log, baik keseluruhan maupun sebagian.

32.2.4. Mendukung penyimpanan log saat proses save game.

32.2.5. Menjadi sumber informasi historis untuk debugging atau tampilan riwayat permainan.

32.3. Relasi

32.3.1. LogEntry (Composition): TransactionLog menyimpan kumpulan LogEntry sebagai riwayat permainan.

32.3.2. GameManager (Association/Composition): TransactionLog digunakan oleh GameManager untuk mencatat setiap aksi yang terjadi dalam permainan.

32.3.3. SaveLoadManager (Dependency): TransactionLog dapat digunakan oleh SaveLoadManager untuk menyimpan dan memuat log ke/dari file.

32.4. Atribut

32.4.1. logs (vector dari LogEntry) : kumpulan seluruh entri log selama permainan

32.5. Metode

32.5.1. recordEvent() : menambahkan entri log baru

32.5.2. getAllLogs() : mengembalikan seluruh riwayat log dalam bentuk string

32.5.3. getRecentLogs(int n) : mengembalikan n log terakhir

32.5.4. clearLogs() : menghapus seluruh log

32.5.5. toSaveFormat() : fungsi helper untuk mengembalikan seluruh log dalam TransactionLog menjadi bentuk string dalam format save file

33. GameException

33.1. Deskripsi

GameException merupakan kelas dasar (base class) untuk seluruh exception dalam sistem permainan. Kelas ini merepresentasikan kesalahan umum yang terjadi selama eksekusi program dan menjadi induk bagi exception yang lebih spesifik.

33.2. Tanggung Jawab

33.2.1. Menyediakan representasi umum untuk error dalam sistem

33.2.2. Menyimpan informasi dasar terkait error (pesan error)

33.2.3. Menjadi parent class bagi seluruh exception turunan

33.3. Relasi

33.3.1. Exception (Inheritance) : Turunan dari kelas std::exception

33.3.2. GameManager (Dependency) : Exception ketika terjadi kesalahan pada program

33.4. Atribut

33.4.1. message : deskripsi error

33.5. Metode

33.5.1. what()

34. InputException

34.1. Deskripsi

InputException merupakan turunan dari GameException yang digunakan untuk menangani kesalahan yang berasal dari input pengguna yang tidak valid atau tidak sesuai format yang diharapkan.

34.2. Tanggung Jawab

34.2.1. Menangani error akibat input tidak valid

34.2.2. Memberikan informasi kesalahan input kepada sistem atau user

34.3. Relasi

34.3.1. GameException (Inheritance) : Turunan dari kelas GameException

35. ConfigException

35.1. Deskripsi

ConfigException merupakan turunan dari GameException yang digunakan untuk menangani kesalahan yang terjadi saat membaca atau memproses file konfigurasi.

35.2. Tanggung Jawab

35.2.1. Menangani error saat parsing file konfigurasi

35.2.2. Menangani format konfigurasi yang tidak valid

35.2.3. Menyampaikan detail kesalahan konfigurasi

35.3. Relasi

35.3.1. GameException (Inheritance) : Turunan dari kelas GameException

35.3.2. Config (Dependency) : Digunakan ketika terjadi kesalahan di config

36. SaveLoadException

36.1. Deskripsi

SaveLoadException merupakan turunan dari GameException yang digunakan untuk menangani kesalahan yang terjadi saat proses penyimpanan (save) atau pemuatan (load) state permainan.

36.2. Tanggung Jawab

36.2.1. Menangani error saat membaca file save

36.2.2. Menangani data save yang rusak atau tidak valid

36.2.3. Menangani kegagalan proses penyimpanan

36.3. Relasi

36.3.1. GameException (Inheritance) : Turunan dari kelas GameException

36.3.2. SaveLoadManager (Dependency) : Digunakan ketika terjadi kesalahan di save atau load

37. GameplayException

37.1. Deskripsi

GameplayException merupakan turunan dari GameException yang digunakan untuk menangani kesalahan yang terjadi selama mekanisme permainan berlangsung.

37.2. Tanggung Jawab

37.2.1. Menangani aksi permainan yang tidak valid

37.2.2. Menangani kondisi permainan yang tidak sesuai aturan

37.2.3. Memberikan informasi kesalahan terkait gameplay

37.3. Relasi

37.3.1. GameException (Inheritance) : Turunan dari kelas GameException

38. GameManager

38.1. Deskripsi

GameManager merupakan kelas pusat yang mengatur seluruh alur permainan. Kelas ini bertanggung jawab dalam mengelola siklus permainan mulai dari inisialisasi, giliran pemain, interaksi dengan board, hingga kondisi akhir permainan seperti kemenangan dan kebangkrutan. GameManager bertindak sebagai pengendali utama yang mengoordinasikan interaksi antar objek seperti Player, Board, Tile, Deck, dan sistem lainnya.

38.2. Tanggung Jawab

38.2.1. Mengatur lifecycle permainan (start, end)

38.2.2. Menjalankan loop turn-based

38.2.3. Menerima dan memproses command pemain

38.2.4. Mengoordinasikan interaksi antar manager

38.2.5. Mengecek kondisi kemenangan / akhir game

38.3. Relasi

38.3.1. TurnManager, Bank (Composition) : Untuk membagi tugas GameManager menjadi beberapa bagian

38.3.2. Player (Aggregation) : GameManager mengelola daftar player

38.3.3. Board (Composition) : GameManager memiliki papan permainan

38.3.4. Deck (Composition) : GameManager memiliki deck kartu

38.3.5. Config, SaveLoad (Dependency) : Dipakai saat initialization dan save atau load game

38.3.6. ViewGame (Association) : Menerima command dari dan mengirim state ke ViewGame

38.4. Atribut

38.4.1. players (vector of Player) : daftar player

38.4.2. board : papan permainan

38.4.3. chance_deck : Deck dari kartu chance

38.4.4. community_chest_deck : Deck dari kartu dana umum

38.4.5. turn_manager : TurnManager

38.4.6. bank : Bank

38.4.7. game_over : status apakah permainan sudah berakhir

38.4.8. max_turn : turn maksimal permainan

38.5. Metode

38.5.1. startGame() : inisialisasi dan setup permainan

38.5.2. endGame() : mengakhiri sesi permainan

38.5.3. processTurn() : menjalankan satu siklus turn permainan

38.5.4. checkWinCondition() : mengecek apakah status kemenangan permainan

38.5.5. handleCommand() : menerima command dari ViewGame

39. EconomyManager

39.1. Deskripsi

Berupa komponen domain ekonomi game yang mengatur arus uang, pajak, lelang, dan penanganan kondisi pemain tidak mampu bayar. Kelas ini fokus pada aturan finansial, bukan parsing file atau tampilan.

39.2. Tanggung Jawab

39.2.1. Menjalankan transaksi dasar antar entitas game (addMoney, deductMoney, transferMoney).

39.2.2. Mengelola proses pajak dan lelang dari awal sampai penentuan pemenang.

39.2.3. Menangani skenario kebangkrutan dan likuidasi kewajiban pemain.

39.3. Relasi

39.3.1. Player (Association/Dependency): memodifikasi saldo dan status finansial pemain.

39.3.2. PropertyTile (Association): dipakai dalam alur lelang dan transfer kepemilikan.

39.3.3. GameManager dan TransactionLog (Dependency): dipanggil dari flow game dan menghasilkan event log ekonomi.

39.4. Atribut

39.4.1. active_bidders : vector Player yang melakukan bidding dalam auction

39.4.2. current_highest_bid : nilai dari bid tertinggi

39.4.3. current_highest_bidder : pointer ke player yang merupakan bidder tertinggi saat itu

39.4.4. bidder_index : index dari player current_highest_bidder

39.4.5. auctioned_property : properti yang dilelangkan dalam auction

39.5. Metode

39.5.1. toSaveFormat() : mengubah state ekonomi ke format string untuk save.

39.5.2. addMoney(Player&, float) : menambahkan uang ke pemain.

39.5.3. deductMoney(Player&, float) : mengurangi uang pemain, return sukses/gagal.

39.5.4. transferMoney(Player&, Player&, float) : memindahkan uang dari payer ke receiver.

39.5.5. processTax(shared_ptr<Player>, TaxType, float) : menghitung dan memproses pembayaran pajak.

39.5.6. startAuction(PropertyTile*) : memulai sesi lelang untuk properti tertentu.

39.5.7. placeBid(float, shared_ptr<Player>&) : mencatat bid baru dari pemain.

39.5.8. foldBid(iterator) : mengeluarkan pemain dari lelang (pass/fold).

39.5.9. isAuctionOver() const : mengecek apakah lelang sudah selesai.

39.5.10. resolveAuction(PropertyTile*, shared_ptr<Player>&) : menyelesaikan lelang dan menetapkan pemenang.

39.5.11. getCurrentBidder() const : mengambil daftar bidder aktif saat ini.

39.5.12. getHighestBid() const : mengambil nilai bid tertinggi saat ini.

39.5.13. isBankruptcyInevitable(Player&, float) const : mengecek apakah hutang pemain tidak mungkin dilunasi.

39.5.14. executeBankruptcy(shared_ptr<Player>, shared_ptr<Player>, float) : mengeksekusi alur kebangkrutan pemain.

40. PropertyManager

40.1. Deskripsi

pengelola papan dan properti. Kelas ini mengorkestrasi status kepemilikan, build/mortgage logic, serta sinkronisasi state properti dari config atau save data.

40.2. Tanggung Jawab

40.2.1. Mengelola objek papan (Board) dan akses tile berdasarkan posisi.

40.2.2. Menangani operasi properti: assign owner, build house/hotel, mortgage/unmortgage, festival.

40.2.3. Melakukan inisialisasi board dari config dan restore state properti dari data save.

40.3. Relasi

40.3.1. Board (Composition): menyimpan instance board statis sebagai pusat tile game.

40.3.2. PropertyTile/StreetTile (Association): objek target utama untuk mutasi state properti.

40.3.3. GameSaveData/PropertySaveData (Dependency): sumber data restore state saat load game.

40.4. Atribut

40.4.1. board : Board yang berisi semua Tile

40.4.2. railroad_rent_map : peta antara jumlah railroad yang dimiliki dengan harga sewa

40.4.3. utility_multiplier_map : peta antara jumlah utility yang dimiliki dengan faktor pengali harga sewa

40.5. Metode

40.5.1. getBoard() : mengambil board global yang dikelola manager.

40.5.2. getTileAt(int) const : mengambil tile pada posisi tertentu.

40.5.3. assignOwnership(PropertyTile*, shared_ptr<Player>) : mengatur kepemilikan properti ke pemain baru.

40.5.4. tryBuildHouse(shared_ptr<Player>, StreetTile*) : validasi lalu build rumah jika aturan terpenuhi.

40.5.5. tryBuildHotel(shared_ptr<Player>, StreetTile*) : validasi lalu build hotel jika aturan terpenuhi.

40.5.6. tryMortgage(shared_ptr<Player>, PropertyTile*) : memproses penggadaian properti jika valid.

40.5.7. tryUnmortgage(shared_ptr<Player>, PropertyTile*) : memproses penebusan properti tergadai.

40.5.8. getFinalRentPrice(PropertyTile*, int) const : hitung harga sewa final (opsional pakai dadu untuk utility).

40.5.9. startFestival(PropertyTile*) : memulai efek festival pada properti.

40.5.10. findPropertiesOwnedByPlayer(shared_ptr<Player>) const : mengambil daftar properti milik pemain.

40.5.11. doMortgage(shared_ptr<Player>) : flow interaktif untuk memilih dan menggadaikan properti.

40.5.12. doUnmortgage(shared_ptr<Player>) : flow interaktif untuk memilih dan menebus properti.

40.5.13. InitializeBoard(FullConfigData&&) : membangun board dari config (tile + map multiplier).

40.5.14. loadBoardState(vector<PropertySaveData>) : restore state properti dari data save.

40.5.15. toSaveFormat() const : serialisasi state properti/board untuk save.

40.5.16. sellAllBuildingsOnColorGroup(shared_ptr<Player>, string) : helper internal untuk menjual bangunan satu color group.

41. CardManager

41.1. Deskripsi

bertugas mengelola seluruh lifecycle kartu: deck kartu aksi, deck skill, proses draw/discard, dan restore state kartu dari save data.

41.2. Tanggung Jawab

41.2.1. Menyimpan dan mengelola deck chance, community chest, dan skill.

41.2.2. Menjalankan alur draw card ke pemain serta discard setelah dipakai.

41.2.3. Memulihkan state kartu dari save (deck + hand cards pemain) melalui fungsi load state.

41.3. Relasi

41.3.1. Deck<T> (Composition): CardManager memiliki beberapa deck sebagai storage utama kartu.

41.3.2. ActionCard/SkillCard (Composition/Dependency): tipe kartu konkret yang diproduksi dan diproses.

41.3.3. Player dan GameSaveData (Association/Dependency): kartu diberikan ke pemain dan direstore dari data save.

41.4. Atribut

41.4.1. chance_deck : deck untuk kartu kesempatan

41.4.2. community_chest_deck : deck untuk kartu dana umum

41.4.3. skill_deck : deck untuk SkillCard

41.5. Metode

41.5.1. initializeDecks() : membaca config dan mengisi deck kartu.

41.5.2. drawKesempatan(Player&) : draw kartu dari deck Kesempatan dan jalankan efeknya.

41.5.3. drawDanaUmum(Player&) : draw kartu dari deck Dana Umum dan jalankan efeknya.

41.5.4. giveSkillCardToPlayer(Player&) : memberikan skill card ke pemain.

41.5.5. takeSkillCardFromPlayer(Player&, int) : mengambil kartu skill dari tangan pemain berdasarkan index.

41.5.6. loadCardState(const GameSaveData&) : restore state deck dan kartu pemain dari save.

41.5.7. discardCard(unique_ptr<...>, Deck<...>&) : memindahkan kartu terpakai ke discard pile deck terkait.

41.5.8. toSaveFormat() const : serialisasi state deck kartu.

41.5.9. createSkillCardFromSave(const CardSaveData&) const : factory internal untuk membuat skill card dari data save.

41.5.10. createSkillDeckCard(const string&) const : factory internal untuk membuat kartu deck dari nama kartu.

42. ConfigData

42.1. Deskripsi

Kelas ini mendefinisikan struktur data untuk menyimpan konfigurasi permainan yang dibaca dari file eksternal yang meliputi informasi properti, tile aksi, batas turn maksimal, dan parameter ekonomi seperti jumlah pajak, jumlah reward_go, balance awal, dll.

42.2. Tanggung Jawab

42.2.1. Menyimpan data konfigurasi setiap properti jalanan

42.2.2. Menyimpan data konfigurasi tile aksi

42.2.3. Mengagregasi seluruh konfigurasi permainan dalam satu struktur

42.3. Relasi

42.3.1. Tile, StreetTile, RailroadTile, UtilityTile, CardTile, TaxTile (Composition): FullConfigData menyimpan kumpulan tile permainan.

42.3.2. IOManager (Dependency): dipakai sebagai hasil parsing config dari file.

42.3.3. GameManager/PropertyManager/EconomyManager (Dependency): jadi sumber data awal untuk inisialisasi board dan rule ekonomi.

42.4. Atribut

42.4.1. streets (vector of Tile) : seluruh Tile property yang ada di dalam permainan

42.4.2. action_tiles (vector of Tile) : seluruh Tile action yang ada di dalam permainan

42.4.3. railroad_rent_map (map of int,int) : memetakan harga sewa berdasarkan jumlah railroad yang dimiliki

42.4.4. utility_multiplier_map (map of int,int) : memetakan faktor pengali untuk harga sewa berdasarkan jumlah Tile Utility yang dimiliki

42.4.5. pph_flat : nilai pajak penghasilan berupa nilai flat

42.4.6. pph_percent : nilai pajak penghasilan berupa persentase

42.4.7. pbm_flat : nilai pajak barang mewah berupa nilai flat

42.4.8. go_salary : nilai yang didapatkan player saat melewati Tile Go

42.4.9. jail_fine : nilai yang harus dibayar player saat masuk penjara

42.4.10. max_turn : batas maksimal turn dalam permainan

42.4.11. initial_balance : nilai balance awal seluruh player saat game dimulai

43. SaveData

43.1. Deskripsi

File ini mendefinisikan struktur data yang merepresentasikan game state dalam format yang dapat disimpan/dimuat, dengan memecahnya menjadi komponen-komponen lebih kecilnya seperti statePlayer, stateProperty, stateLog, stateCard menjadi data struct sendiri.

43.2. Tanggung Jawab

43.2.1. Merepresentasikan elemen game dalam format saveConfig

43.2.2. Menyediakan container untuk seluruh state game dan memudahkan loading

43.3. Relasi

43.3.1. CardSaveData, PlayerSaveData, PropertySaveData, LogSaveData ke GameSaveData (Composition): GameSaveData adalah container utama seluruh state save.

43.3.2. IOManager (Association): dipakai IOManager untuk serialisasi/deserialisasi.

43.3.3. Player, PropertyTile, SkillCard/Deck, TransactionLog, GameManager (Dependency): jadi format perantara (DTO) antara class domain dan file save.

43.4. Atribut

43.4.1. current_turn : value turn saat ini

43.4.2. max_turn : batas maksimal turn dalam permainan

43.4.3. player_count : jumlah player dalam permainan

43.4.4. players (vector of PlayerSaveData) : seluruh data config state player

43.4.5. turn_order (vector of String) : urutan main permainan berisi username dari player

43.4.6. current_active_player : player yang melakukan aksi/jalan/melakukan turn pada saat ini

43.4.7. properties (vector of PropertySaveData) : seluruh data state seluruh property Tile

43.4.8. skill_card_deck (vector of String) : berisi seluruh nama skillCard yang ada di dalam deck pickup

43.4.9. logs (vector of LogSaveData) : berisi seluruh data log

43.5. Metode

44. CardSaveData

44.1. Deskripsi

Kelas data kecil untuk merepresentasikan 1 skill card saat proses save/load, bukan untuk menjalankan logic effect kartu di game.

44.2. Tanggung Jawab

44.2.1. Menyimpan card_type.

44.2.2. Menyimpan value.

44.2.3. Menyimpan sisa durasi efek kartu.

44.3. Relasi

44.3.1. PlayerSaveData (Composition): jadi elemen di hand_cards.

44.3.2. IOManager (Association): dibaca/ditulis saat parsing file save.

44.3.3. SkillCard/Effect domain class (Dependency): sebagai bentuk data transfer dari objek kartu asli.

44.4. Atribut

44.4.1. card_type : berupa nama tipe skillcard

44.4.2. value : berupa nilai yang digunakan beberapa tipe seperti discount untuk persentase discount dan move cards untuk jumlah langkah

44.4.3. remaining_duration : nilai untuk merepresentasikan sisa durasi turn untuk effect card

44.5. Metode

45. PlayerSaveData

45.1. Deskripsi

Kelas data kecil untuk menyimpan snapshot kondisi 1 pemain saat proses save/load.

45.2. Tanggung Jawab

45.2.1. Menyimpan identitas dan status pemain.

45.2.2. Menyimpan state ekonomi dan posisi dalam board.

45.2.3. Menyimpan daftar kartu yang sedang dipegang.

45.3. Relasi

45.3.1. CardSaveData (Composition): memiliki koleksi kartu tangan.

45.3.2. GameSaveData (Composition): jadi elemen di daftar players.

45.3.3. Player domain class (Dependency): representasi data simpannya.

45.4. Atribut

45.4.1. username : nama dari player dalam bentuk string

45.4.2. balance : jumlah uang yang dimiliki player

45.4.3. position_code : berupa kode Tile dalam board yang merepresentasikan posisi Player pada saat ini

45.4.4. status : status player, bisa berupa FREE, INJAIL, atau BANKRUPT

45.4.5. hand_cards (vector of CardSaveCard) : berupa data skillcard yang dimiliki player

45.5. Metode

46. PropertySaveData

46.1. Deskripsi

Kelas data kecil untuk menyimpan kondisi 1 properti pada saat save/load game.

46.2. Tanggung Jawab

46.2.1. Menyimpan identitas properti.

46.2.2. Menyimpan kepemilikan dan status properti.

46.2.3. Menyimpan state dinamis properti seperti festival_multiplier, festival_duration, build_level.

46.3. Relasi

46.3.1. GameSaveData (Composition): jadi elemen di daftar properties.

46.3.2. PropertyTile/StreetTile/RailroadTile/UtilityTile (Dependency): bentuk data transfer dari objek properti nyata.

46.3.3. IOManager (Association): diparse dari dan ke file save.

46.4. Atribut

46.4.1. tile_code : kode singkat dari nama properti

46.4.2. type : tipe dari properti, dapat berupa STREET, RAILROAD, atau UTILITY

46.4.3. owner_name : nama dari pemilik properti, BANK jika tidak dimiliki player manapun

46.4.4. status : status dari properti, bisa berupa BANK, MORTGAGED, atau OWNED

46.4.5. festival_multiplier : faktor pengali dari festival jika ada

46.4.6. festival_duration : durasi turn dari effect festival

46.4.7. build_level : jumlah bangunan pada properti, maks 4 rumah. Value 5 menandakan sebuah Hotel

46.5. Metode

47. LogSaveData

47.1. Deskripsi

Kelas data kecil untuk merepresentasikan 1 entri riwayat aksi game dalam format simpan.

47.2. Tanggung Jawab

47.2.1. Menyimpan nomor turn.

47.2.2. Menyimpan pelaku aksi berupa username dan tipe aksi.

47.2.3. Menyimpan deskripsi dari log.

47.3. Relasi

47.3.1. GameSaveData (Composition): jadi elemen di daftar logs.

47.3.2. TransactionLog/LogEntry (Dependency): versi data transfer dari sistem log runtime.

47.3.3. IOManager (Association): dipakai saat serialisasi dan deserialisasi log.

47.4. Atribut

47.4.1. turn : angka turn saat log ini dibuat

47.4.2. username : nama dari pelaku aksi

47.4.3. action_type : tipe aksi yang dilakukan

47.4.4. detail : deskripsi aksi yang dilakukan player

47.5. Metode

48. IOManager

48.1. Deskripsi

Kelas yang mengelola operasi input/output untuk membaca file konfigurasi dan file save game, serta menulis file save game dari state permainan saat ini.

48.2. Tanggung Jawab

48.2.1. Memuat dan memparse file konfigurasi game dari folder config

48.2.2. Memuat dan memparse file save game untuk restore state permainan

48.2.3. Menyimpan state permainan saat ini ke file save

48.3. Relasi

48.3.1. SaveData (Dependency): IOManager mengisi dan membaca GameSaveData saat load/save.

48.3.2. ConfigData (Dependency): IOManager membentuk FullConfigData saat baca file config.

48.3.3. Tile dan turunannya (Dependency): IOManager membuat objek tile konkret dari data teks config.

48.3.4. Sistem game (Association): jadi jembatan antara file eksternal dan state game di memori.

48.4. Atribut

48.5. Metode

48.5.1. static loadGameData(string& filepath) : melakukan load dari seluruh save file eksternal game state, menghasilkan struct GameSaveData

48.5.2. static saveGameData(GameSaveData& data, string& filepath) : menyimpan seluruh state saat ini ke dalam file eksternal

48.5.3. static loadAllConfigs(string& configDirectoryPath) : melakukan load dari seluruh file config, menghasilkan struct FullConfigData

48.5.4. IOparsePropertyTileConfig(string& configDirectoryPath) : method helper untuk parse data properti

48.5.5. IOparseActionTileConfig(string& configDirectoryPath, vector of int (tax), vector of int (special)) : method helper untuk parse data ActionTile

48.5.6. IOparseTaxConfig(string& configDirectoryPath) : method helper untuk parse data tax (pph dan pbm)

48.5.7. IOparseSpecialConfig(string& configDirectoryPath) : method helper untuk parse data Tile spesial (go_salary dan jail_fine)

48.5.8. IOparseMiscConfig(string& configDirectoryPath) : method helper untuk parse data misc (max_turn dan initial_balance)

48.5.9. IOparseRailroadConfig(string& configDirectoryPath) : method helper untuk parse parse data map harga sewa railroad

48.5.10. IOparseUtilityConfig(string& configDirectoryPath) : method helper untuk parse data map multiplier harga sewa utility

49. ViewGame

49.1. Deskripsi

ViewGame adalah komponen antarmuka (View) yang bertanggung jawab untuk menangani seluruh input dari pengguna dan menampilkan output informasi permainan ke layar. Kelas ini bersifat stateless (menggunakan metode statis) untuk memisahkan logika presentasi dari logika bisnis permainan.

49.2. Tanggung Jawab

49.2.1. Mengambil dan memvalidasi input dasar dari pengguna (perintah, nama, angka, pilihan Yes/No).

49.2.2. Menampilkan visualisasi papan permainan dan hasil kocokan dadu.

49.2.3. Menyajikan informasi mendetail mengenai properti (akta, daftar kepemilikan, status gadai).

49.2.4. Menampilkan dialog interaksi transaksi seperti pembelian, pembayaran sewa, pajak, dan lelang.

49.2.5. Mengelola tampilan visual untuk kejadian khusus seperti Festival, kartu *Chance/Community Chest*, dan penggunaan *Skill Card*.

49.2.6. Menampilkan status krusial seperti proses likuidasi aset, kebangkrutan, serta pengumuman pemenang.

49.2.7. Menangani pesan kesalahan (exception) dan log aktivitas permainan.

49.3. Relasi

49.3.1. Player (Dependency): Mengakses data pemain untuk menampilkan saldo dan status.

49.3.2. PropertyTile / StreetTile (Dependency): Mengambil informasi detail petak untuk ditampilkan kepada pemain.

49.3.3. SkillCard (Dependency): Menampilkan daftar dan efek kartu skill yang dimiliki atau digunakan.

49.3.4. LogEntry (Dependency): Menampilkan catatan riwayat permainan.

49.3.5. GameException (Dependency): Menangkap dan menampilkan pesan kesalahan logika game.

49.4. Atribut

49.5. Metode

49.5.1. getUserCommand() / getUserInput(): Mengambil input string dari pengguna.

49.5.2. getInt(max_int): Mengambil input angka dengan batasan tertentu.

49.5.3. displayBoard(): Menampilkan status papan permainan saat ini.

49.5.4. displayDiceRollResult(...): Menampilkan hasil lemparan dadu dan lokasi mendarat.

49.5.5. displayPropertyDeed(code): Menampilkan rincian akta properti tertentu.

49.5.6. displayRentPayment(...): Menampilkan rincian pembayaran sewa antar pemain.

49.5.7. displayAuction...(): Serangkaian metode untuk mengelola tampilan proses lelang.

49.5.8. displayLiquidationPanel(...): Menampilkan opsi bagi pemain yang terancam bangkrut untuk menjual aset.

49.5.9. displayCardDraw(...): Menampilkan teks kartu yang diambil dari petak keberuntungan.

49.5.10. displaySave/Load...(): Menampilkan status keberhasilan atau kegagalan penyimpanan data.

49.5.11. displayException(e): Menampilkan pesan kesalahan sistem secara rapi kepada pengguna.

3. Justifikasi Desain

Desain ini dapat menerapkan konsep konsep dalam Object Oriented Programming seperti polymorphism, abstraction, encapsulation, dan inheritance. Fokus dari desain ini adalah *single responsibility* yang menjamin modularity yang baik. Desain ini juga menerapkan konsep lainnya seperti template dan generic serta error handling yang baik. Karena itu, kami memilih desain oop ini dalam pembuatan game monopoli ini agar development bisa dilakukan secara modular.

Namun, desain ini memiliki kekurangan yaitu jumlah class yang sangat banyak, terutama untuk implementasi abstract class seperti card, tile, dan GameException. Namun desain ini jauh lebih preferable sebagai bentuk perwujudan *Object Oriented Design*. Selain itu, kelebihan utama dari desain ini yaitu memiliki banyak pilihan *range* mulai dari perubahan spesifik hingga perubahan yang sangat umum dan semua skala perubahan di antara spesifik dan umum itu karena banyaknya *abstract class* yang menciptakan hierarki yang sangat jelas.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

4.1.1. Tile

Konsep inheritance diterapkan melalui kelas Tile sebagai kelas dasar yang mendefinisikan properti dasar (seperti indeks, nama, kode, dan warna) serta metode virtual murni. Kelas Tile sendiri digunakan sebagai cetak biru untuk setiap petak yang ada di dalam papan permainan. Dengan menggunakan konsep Inheritance, setiap petak yang memiliki fungsi berbeda (seperti petak pajak, properti, atau penjara) dapat berbagi atribut dasar yang sama namun tetap memiliki perilaku khusus pada metode tertentu.

```
class Tile {
protected:
    int index;
    std::string name;
    std::string code;
    std::string color;

public:
    Tile(int index, std::string name, std::string code, std::string color);
    virtual ~Tile() = default;

    // Metode virtual yang akan diimplementasikan secara spesifik oleh kelas turunan
    virtual void onLand(Player& p) = 0;
    virtual std::string getStatusString() const { return ""; }
    virtual bool isProperty() const { return false; }
    virtual bool isStreet() const { return false; }
};
```

Kelas ini diturunkan menjadi berbagai kelas spesifik yaitu CardTile, TaxTile, FestivalTile, GoTile, GoToJailTile, FreeParkingTile, JailTile, dan PropertyTile. Selanjutnya, PropertyTile juga diturunkan kembali menjadi StreetTile, RailroadTile, dan UtilityTile. Contoh pada kelas StreetTile:

```
class StreetTile : public PropertyTile {
private:
    int building_level;
    std::vector<float> rent_list;
    float house_price;
    float hotel_price;

public:
    StreetTile(/* parameter constructor */);

    // Meng-override perilaku dasar untuk kebutuhan spesifik jalan (street)
    float calculateRent() const override;
    void onLand(Player& p) override;
    PropertyType getPropertyType() const override;

    bool isStreet() const override { return true; }
    std::string getStatusString() const override;
};
```

Pada kelas ini, StreetTile mewarisi seluruh atribut dari PropertyTile dan Tile, namun menambahkan spesifikasi untuk manajemen bangunan (rumah/hotel) dan daftar harga sewa. Constructor kelas ini memanggil constructor base (PropertyTile) untuk menginisialisasi status kepemilikan dan harga beli, kemudian menginisialisasi atribut mandiri seperti rent_list dan building_level. Konsep polymorphism pada kelas ini digunakan pada berbagai komponen sistem seperti papan permainan (Board) dan pengelola giliran (Turn/Game Manager) menggunakan method onLand()

```
// CardTile
CardTile::CardTile(int index, std::string name, std::string code, std::string color, CardType card)
    : Tile(index, name, code, color), card(card) {}
CardType CardTile::getType(){
    return card;
}
void CardTile::onLand(Player& p) {
    GameManager::visitCardTile(this, p);
}

// TaxTile
TaxTile::TaxTile(int index, std::string name, std::string code, std::string color, TaxType tax_type, float tax_amount)
```

```
    :  Tile(index, name, code, color), tax_type(tax_type), tax_amount(tax_amount) {}
float TaxTile::getTaxAmount(){
    return tax_amount;
}
void TaxTile::onLand(Player& p) {
    GameManager::visitTaxTile(this, p);
}
```

4.1.2. Card

Konsep inheritance diterapkan melalui kelas Card sebagai kelas dasar (base class) yang mendefinisikan kontrak utama bagi seluruh jenis kartu dalam permainan. Kelas ini menyediakan metode virtual murni onDraw(Player& p) yang memastikan bahwa setiap jenis kartu, baik itu kartu aksi yang diambil dari petak keberuntungan maupun kartu skill yang disimpan pemain, memiliki perilaku saat diaktifkan. Penggunaan inheritance di sini memungkinkan sistem untuk mengelola berbagai jenis logika kartu (perpindahan, finansial, atau perlindungan) dalam satu hierarki yang terorganisir.

```
class Card
{
protected:
    static std::random_device rd;
    static std::mt19937 gen;
public:
    Card();
    virtual ~Card() = default;
    // Kontrak utama: setiap kartu harus mendefinisikan apa yang terjadi saat ditarik
    virtual void onDraw(Player& p) = 0;
};
```

Kelas ini diturunkan menjadi dua cabang utama: ActionCard (untuk kartu Chance/Community Chest) dan SkillCard (untuk kartu kemampuan pemain). ActionCard kemudian diturunkan lagi menjadi FinancialActionCard, JailActionCard, MoveActionCard, dan StationActionCard. Sementara itu, SkillCard diturunkan menjadi MoveSkillCard, DiscountSkillCard, ShieldSkillCard, TeleportSkillCard, LassoSkillCard, dan DemolitionSkillCard. Contoh pada kelas SkillCard dan turunannya MoveSkillCard:

```
class SkillCard : public Card {
protected:
    std::string name;
public:
    SkillCard(std::string name);
    void onDraw(Player& p) override; // Implementasi dasar onDraw untuk Skill
    virtual void useEffect(Player& p) = 0; // Metode virtual murni khusus Skill

    // Virtual helper untuk identifikasi tipe tanpa casting manual yang berat
    virtual bool isMoveCard() const { return false; }
};

class MoveSkillCard : public SkillCard {
private:
    int step;
public:
    bool isMoveCard() const override { return true; }
    void useEffect(Player& p) override; // Implementasi spesifik efek gerakan
};
```

Pada hierarki ini, SkillCard bertindak sebagai kelas antara yang menambahkan atribut name dan metode useEffect. Kelas spesifik seperti MoveSkillCard kemudian mengimplementasikan logika pergerakan yang unik. Constructor pada kelas turunan akan meneruskan nama skill ke constructor SkillCard, sementara logika internal seperti jumlah langkah (step) dikelola secara mandiri oleh kelas tersebut.

Konsep polymorphism pada kelas ini sangat krusial saat menangani koleksi kartu atau eksekusi efek secara dinamis. Dengan polimorfisme, Game Manager dapat memproses berbagai kartu tanpa perlu mengetahui tipe spesifiknya secara eksplisit di setiap baris kode. Berikut adalah contoh potongan kode yang mengimplementasikan konsep tersebut:

```
```

4.1.3. Save

4.2. Method/Operator Overloading

4.2.1. Player

Konsep Operator Overloading diterapkan melalui kelas Player. Penggunaan Operator overloading pada kelas Player dilakukan agar dapat memberikan operasi yang bermakna dan kejelasan saat melakukan transaksi perpindahan uang yang dimiliki oleh player. Penggunaan operator memberikan sistem kemudahan untuk melakukan penambahan jumlah uang ataupun pengurangan jumlah uang dari player

```
class Player {
     public:
               Player& operator+=(float amount);
               Player& operator-=(float amount);
               Player operator+(float amount) const;
               Player operator-(float amount) const;
}
```

Pada overloading function tersebut, ketika player melakukan operasi addition dan subtraction dengan sebuah float. Uang yang dimiliki oleh player akan berubah sejumlah float. Hal tersebut membuat kode lebih mudah dibaca dan lebih mudah dimengerti dengan operasi penjumlahan dan pengurangan yang bermakna dengan tipe data float.

4.3. Template & Generic Classes

4.3.1. Deck

Konsep Template dan Generic Class diterapkan oleh kelas Deck. Deck merupakan representasi dari tumpukan kartu yang Action Card seperti kartu dana umum dan kartu kesempatan. Deck juga merupakan representasi dari tumpukan kartu skill card yang dapat diambil saat memulai turn. Penggunaan Template dan Generic class pada deck membuat deck bisa menampung dua jenis tiba card sehingga mempermudah pengembangan dan membuat kode lebih simpel. Berikut merupakan Cuplikan kode

```
template <class T>
class Deck : public Saveable {
private:
    std::vector<T> cards;
    std::vector<T> discard;
public:
    T drawCard()
    void shuffle()
    void addToDiscard()
}
```

Pada kelas Deck tersebut terdapat vector<T> yang merepresentasikan sejumlah kartu pada tumpukan. Dengan penggunaan Template dan Generic Class seperti berikut implementasi deck dapat mudah digunakan untuk dua jenis object yang berbeda yaitu ActionCard dan SkillCard.

4.4. Exception

Exception adalah tipe data khusus yang mengembalikan suatu nilai saat terjadi kesalahan dalam proses run program. Exception akan dikembalikan dengan suatu keyword “throw” berdasarkan suatu alur yang sudah didefinisikan, dan saat terjadi suatu kesalahan yang sesuai dengan definisi Exception tersebut, suatu objek Exception yang dilempar tersebut harus ditangkap (catch) agar proses run program tidak diberhentikan karena error. Hal ini disebut sebagai error handling. Exception yang dibuat antara lain:

4.4.1. SaveLoadException

Exception yang dibuat untuk melakukan error handling pada proses save dan load permainan

```
void SaveLoadManager::save(const GameManager& g, const std::string& filepath) {
    std::ofstream outFile(filepath);


    if (!outFile.is_open()) {
        throw SaveLoadException("Could not open or create file at " + filepath + "\n");
```

```
        return;
    }
    std::string saveData = g.toSaveFormat();

    outFile << saveData;
    outFile.close();

    std::cout << "Game successfully saved to " << filepath << "!\n";
}
```

4.4.2. GameplayException - handling selama gameplay berlangsung

Exception yang dibuat untuk melakukan error handling selama game atau permainan berlangsung

```
class Deck : public Saveable {
private:
    std::vector<T> cards;
    std::vector<T> discard;

public:
    void setCards(std::vector<T>&& newCards) {
        cards = std::move(newCards);
    }

    void clearDiscard() {
        discard.clear();
    }

    T drawCard() {
        if (cards.empty()) {
            throw GameplayException("Deck: Cannot draw from an empty deck!");
        }

        T temp = std::move(cards.back());
        cards.pop_back();
        return temp;
    }
```

4.5. C++ Standard Template Library

4.5.1. std::vector

Vector digunakan di program kami untuk mengumpulkan koleksi dari beberapa object yang sama agar dapat mempermudah akses dan manipulasi data dari kumpulan objek yang sama.

4.5.1.1. Player

```
std::vector<std::unique_ptr<SkillCard>> saved_cards;
std::vector<std::unique_ptr<Effect>> active_effects;
std::vector<PropertyTile*> owned_properties;
```

4.5.1.2. GameManager

```
std::vector<PropertyTile*> owned_properties;
```

4.5.1.3. StreetTile

```
std::vector<float> rent_list;
```

4.5.1.4. Deck

```
std::vector<T> cards;
std::vector<T> discard;
```

4.5.1.5. Board

```
std::vector<std::unique_ptr<Tile>> tiles;
```

4.5.1.6. ConfigData

```
std::vector<int> rent_prices;
std::vector<std::unique_ptr<Tile>> streets;
std::vector<std::unique_ptr<Tile>> action_tiles;
```

4.5.1.7. SaveData

```
std::vector<PlayerSaveData> players;
std::vector<std::string> turn_order;
std::vector<PropertySaveData> properties;
std::vector<std::string> skill_card_deck;
std::vector<LogSaveData> logs;
```

4.5.1.8. EconomyManager

```
std::vector<std::shared_ptr<Player>> active_bidders;
```

4.5.2. std::map

Penggunaan map pada program kami dilakukan untuk mempermudah melakukan pencarian dengan sebuah indeks yang dipetakan kepada sebuah value. Hal tersebut mempermudah pencarian agar tidak perlu menerapkan data struktur map sendiri dan menggunakan data struktur map standard template library yang sudah RAII sehingga dapat mengatasi masalah alokasi dan dealokasi resource

4.5.2.1. ConfigData

```
std::map<int, int> railroad_rent_map;
std::map<int, int> utility_multiplier_map;
```

4.5.2.2. RailroadTile

```
static const std::map<int, int> railroad_multiplier;
```

4.5.2.3. UtilityTile

```
static const std::map<int, int> utility_multiplier;
```

4.5.3. std::unordered_map

Penggunaan Unordered map pada program kami dilakukan untuk mempermudah melakukan pencarian dengan sebuah value misal string yang dipetakan ke sebuah value. Hal tersebut mempermudah pencarian agar tidak perlu menerapkan data struktur map sendiri dan menggunakan data struktur map stl yang sudah RAII. Pencarian menggunakan unordered_map juga cepat karena

4.5.3.1. Board

```
std::unordered_map<std::string, std::vector<Tile*>> colorTileMap;
```

4.5.3.2. GameManager

```
std::unordered_map<std::string, std::function<void(const std::string&)>> command_map;
```

4.5.4. std::shared_ptr

shared_ptr merupakan pointer pintar yang dapat menyimpan informasi dari jumlah penunjuk ke dirinya. Kegunaan shared_ptr adalah pada kemudahan untuk mengurus alokasi resource bila jumlah penunjuk pada pointer menjadi 0, maka akan melakukan dealokasi object yang tertunjuk.

4.5.4.1. EconomyManager

```
std::vector<std::shared_ptr<Player>> active_bidders;
std::shared_ptr<Player> current_highest_bidder;
```

4.5.4.2. GameManager

```
static std::vector<std::shared_ptr<Player>> players;
```

4.5.5. std::unique_ptr

unique_ptr, seperti shared_ptr merupakan sebuah pointer yang pintar, unique_ptr dapat melakukan penghapusan objek yang ditunjuk bila sudah selesai digunakan dan tidak menyebabkan memory leak. Keunikan dari unique_ptr adalah menjamin bahwa tidak ada dua bagian program yang mencoba menghapus objek yang sama. Jika kamu mencoba menyalin (copy) sebuah unique_ptr, compiler akan memberikan error. Objek hanya bisa dipindahkan (std::move), bukan digandakan.

4.5.5.1. GameManager

```
static std::unique_ptr<CardManager> card_manager;
static std::unique_ptr<PropertyManager> property_manager;
static std::unique_ptr<EconomyManager> economy_manager;
static std::unique_ptr<TransactionLog> logger;
```

4.5.5.2. PropertyManager

```
static std::unique_ptr<Board> board;
```

4.5.5.3. Board

```
std::vector<std::unique_ptr<Tile>> tiles;
```

4.5.5.4. ConfigData

```
std::vector<std::unique_ptr<Tile>> streets;
std::vector<std::unique_ptr<Tile>> action_tiles;
```

4.5.5.5. CardManager

```
Deck<std::unique_ptr<Card>> chance_deck;
Deck<std::unique_ptr<Card>> community_chest_deck;
Deck<std::unique_ptr<Card>> skill_deck;
```

4.5.5.6. Player

```
   std::vector<std::unique_ptr<SkillCard>> saved_cards;
   std::vector<std::unique_ptr<Effect>> active_effects;
```

4.5.6. Std::stringstream

stringstream

4.5.6.1. IOManager

```
   std::getline(file,line);
   std::istringstream header1(line);
   header1 >> res.current_turn >> res.max_turn;
   std::getline(file,line);
   std::istringstream header2(line);
   header2 >> res.player_count;
```

4.5.6.2. GameManager

```
std::string GameManager::toSaveFormat() const{
    std::ostringstream out;
    out << current_turn_count << " " << max_turns << "\n";
    out << players.size();
    for(const std::shared_ptr<Player>& p:players){
        out << p->toSaveFormat();
    }
    for(int i = 0; i<current_turn_count; i++){
        if(i!=current_turn_count-1) out << players[i]->getname() << " ";
        else out << players[i]->getname() << "\n";
    }
    out << players[players.size()%current_turn_count]->getname() << "\n";
    return out.str();
}
```

4.5.6.3. CardManager

```
std::string CardManager::toSaveFormat() const{
    std::ostringstream out;
```

```
    out << chance_deck.toSaveFormat();
    out << community_chest_deck.toSaveFormat();
    out << skill_deck.toSaveFormat();
    return out.str();
}
```

4.5.6.4. GameManager

```
std::string GameManager::toSaveFormat() const{
    std::ostringstream out;
    out << current_turn_count << " " << max_turns << "\n";
    out << players.size();
    for(const std::shared_ptr<Player>& p:players){
        out << p->toSaveFormat();
    }
    for(int i = 0; i<current_turn_count; i++){
        if(i!=current_turn_count-1) out << players[i]->getname() << " ";
        else out << players[i]->getname() << "\n";
    }
    out << players[players.size()%current_turn_count]->getname() << "\n";
    return out.str();
}
```

4.5.6.5. Player

```
std::string Player::toSaveFormat() const {
    std::ostringstream out;
    out << name << " " << balance << " " << position;
    switch (player_state)
    {
    case PlayerState::FREE:
        out<<"FREE\n";
        break;
    case PlayerState::INJAIL:
        out<<"INJAIL\n";
        break;
    case PlayerState::BANKCRUPT:
        out<<"BANKRUPT\n";
        break;
    }
```

```
    out << saved_cards.size() << "\n";
    for(const std::unique_ptr<SkillCard>& card : saved_cards){
        out << card->toSaveFormat();
    }
    return out.str();
}
```

4.5.6.6. SkillCard

```
std::string SkillCard::toSaveFormat() const{
    std::ostringstream out;
    out << name << "\n";
    return out.str();
}
```

4.5.7. std::algorithm

4.5.7.1. Find_if

```
   auto p = std::find_if(owned.begin(), owned.end(), [input](PropertyTile* n){
        return n->getCode() = input;
    });
```

4.5.7.2. remove

5.

```
void Player::removeProperty(PropertyTile* property){
    owned_properties.erase(
        std::remove(owned_properties.begin(), owned_properties.end(), property),
        owned_properties.end()
    );
}
```

5.1.1.1. sort

```
std::sort(
     properties_to_sell.begin(),
     properties_to_sell.end(),
     [](PropertyTile* a, PropertyTile* b) {
         return a->getBuyPrice() < b->getBuyPrice();
     }
 );
```

5.1.1.2. remove_if

```
    std::remove_if(
         active_effects.begin(),
         active_effects.end(),
         [](const std::unique_ptr<Effect>& effect) {
             return effect->isExpired();
         }
     ),
     active_effects.end()
 );
```

5.1.1.3. count_if

```
return std::count_if(owned_properties.begin(), owned_properties.end(),
     [](PropertyTile* prop) {
         return prop->getPropertyType() == PropertyType::RAILROAD;
     });
```

5.1.1.4. shuffle

```
    std::shuffle(discard.begin(), discard.end(), g);
```

5.1.1.5. max

```
int start_idx = std::max(0, size - n);
```

5.2. Composition

5.2.1. Board - Tile

```
std::vector<std::unique_ptr<Tile>> tiles;
```

5.2.2. TransactionLog - LogEntry

```
std::vector<LogEntry> logs;
```

5.2.3. IOManager - SaveData & ConfigData

```
static GameSaveData loadGameData(const std::string& filepath);
static FullConfigData loadAllConfigs(const std::string& configDirectoryPath);
```

5.3. Association

5.3.1. Player - PropertyTile

5.3.2. GameManager - Player

5.4. Abstract Base Class

5.4.1. Effect

Effect merupakan abstract base class yang digunakan untuk merepresentasikan effect yang sedang dimiliki user. Alasan kami memilih ini dibanding penerapan langsung di Player adalah agar effect yang ada bisa diextend tanpa mengubah class Player.

5.4.2. Saveable

Saveable merupakan abstract base class yang digunakan untuk melakukan saving agar object bisa diformat menjadi bentuk yang bisa di-save. Kenapa perlu? Ya biar bisa di-save.

5.4.3. Card

Card merupakan abstract base class dari implementasi semua card yang ada, yang mana tujuan utama kegunaannya adalah untuk polymorphism.

5.4.4. ActionCard

ActionCard merupakan turunan dari card yang sekaligus abstract base class dari implementasi semua card yang langsung dieksekusi ketika di-draw (diambil dari deck).

5.4.5. SkillCard

SkillCard merupakan turunan dari card yang sekaligus abstract base class dari implementasi semua card kemampuan yang bisa disimpan dulu untuk kemudian bisa dipakai user.

6. Bonus Yang dikerjakan

Tidak Ada

7. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Board	13524024, 13524082	13524082
PropertyTile	13524058	13524082
ActionTile	13524058	13524082
SpecialTile	13524058	13524082
Dadu dan Giliran Bermain	13524058, 13524082	13524082
Kepemilikan Properti	13524082, 1352083	13524082
Ketentuan Sewa	13524083	13524082
Mortgage	13524083	13524082
Peningkatan Properti	13524083, 13524082	13524082
Auction	13524082	13524082
Tax	13524082	13524082
Festival	13524058	13524082
Chance and Community Chest Cards	13524024, 13524082	13524082
SkillCards	13524024, 13524082	13524082

Bankruptcy	13524024	13524082
Pengambilalihan Asset	13524082	13524082
Kondisi Game Selesai	13524058	13524082
Save/Load	13524072, 13524024	13524072, 13524024
Transaction Logger	13524072	13524072

Lampiran

Lampiran A. Form Asistensi

IF2010_TB1_Asistensi_TSR